

1º DAW - Entornos de Desarrollo

Documentación con Javadocs

Usando Javadocs

Crear documentación de java para nuestros proyectos es de vital importancia, ya que lo que hagamos tenemos que documentarlo.

Esto va a servir a otros programadores a entender de una forma fácil todo lo que se hizo en un proyecto y saber para qué sirve cada clase, interface y cada método.

La documentación que realizaremos será muy parecida a la documentación oficial javaDoc y será accedida de igual forma desde un navegador web.

Para crear documentación javaDoc de nuestro proyecto, los editores de Java como son: Eclipse, NetBeans, etc, cuentan con opciones de menú que ejecutan la herramienta de creación de documentación de nuestros proyectos.

Crear documentación de Java en Netbeans – javaDoc

Para generar la documentación de java se puede hacer de dos formas:

1. Desde nuestro Editor preferido.
2. Desde línea de comandos.

Independientemente de la opción que escojamos, se usará la herramienta “javadoc” de nuestro JDK.

La documentación se va a generar en formato HTML de forma local dentro de nuestro proyecto.

Para poder generar la documentación de alguna clase o método se tienen que usar etiquetas HTML precedidas del carácter “@”. Las etiquetas tienen que estar dentro de un comentario java iniciando con “/**” y terminando con “*/”.

Estos comentarios con etiquetas de documentación pueden ir al inicio de:

1. Una clase.
2. Un atributo.
3. Un método.

Etiquetas javaDoc

A continuación, veremos una lista de las etiquetas más comunes para generar JavaDoc:

Etiqueta	Descripción
@author	Indica el nombre de quien desarrolló el componente.
@deprecated	Indica que algún método o clase u otro componente está obsoleto.
@param	Indica un parámetro de un método, se tiene que usar para todos los parámetros del método.
@return	Indica el valor de retorno de un método
@see	Indica que el componente puede hacer referencia a otro. Ejemplo: #método(); clase#método();
@throw	Indica que excepción lanza el método.
@version	Indica la versión actual del componente.

Ya que sabemos algunas de las etiquetas más importantes para crear documentación en Java, el paso siguiente es agregar los comentarios con dichas etiquetas en nuestras clases, interfaces y métodos.

Ejemplo de javaDoc

Como ejemplo vamos a usar la siguiente clase Empleado.

```
package empleado;

/**
 * Esta clase contiene los atributos y metodos de un empleado
 * @author Profesor
 * @version 1.0
 */
public class Empleado{

    private int numeroEmpleado;
    private String nombre;
    private int edad;
    private char sexo;
    private String departamento;
    private String puesto;

    /**
     * Metodo constructor por defecto
     */
    public Empleado(){
    }

    /**
     * Metodo constructor parametrizado
     * @param nombre Nombre del empleado
     * @param edad Edad del empleado
     * @param sexo Sexo del empleado en formato H o M
     * @param numeroEmpleado Numero de empleado
     * @param departamento Departamento donde trabajara el empleado
     * @param puesto Puesto que ocupara el empleado dentro de la empresa
     */
    public Empleado(String nombre,
                     int edad,
                     char sexo,
                     int numeroEmpleado,
                     String departamento,
                     String puesto){
        this.nombre = nombre;
```

```

    this.edad = edad;
    this.sexo = sexo;
    this.numeroEmpleado = numeroEmpleado;
    this.departamento = departamento;
    this.puesto = puesto;
}

/**
 * Metodo para regresar el nombre de empleado
 * @return Regresa el nombre del empleado
 */
public String getNombre() {
    return nombre;
}

/**
 * Establece el nombre del empleado
 * @param nombre String que se le asignara al empleado
 */
public void setNombre(String nombre) {
    this.nombre = nombre;
}

/**
 * Metodo para regresar la edad del empleado
 * @return Regresa la edad del empleado
 */
public int getEdad() {
    return edad;
}

/**
 * Establece la edad del empleado
 * @param edad Numero que se le asignara al empleado
 */
public void setEdad(int edad) {
    this.edad = edad;
}

/**

```

```

    * Metodo para regresar el sexo de empleado
    * @return Regresa el sexo del empleado
    */
    public char getSexo() {
        return sexo;
    }

    /**
     * Establece el sexo del empleado
     * @param sexo Char que se le asignara al empleado
     */
    public void setSexo(char sexo) {
        this.sexo = sexo;
    }

    /**
     * Metodo para regresar el numero de empleado
     * @return Regresa el numero del empleado
     */
    public int getNumeroEmpleado() {
        return numeroEmpleado;
    }

    /**
     * Establece el numero del empleado
     * @param numeroEmpleado Numero que se le asignara al empleado
     */
    public void setNumeroEmpleado(int numeroEmpleado) {
        this.numeroEmpleado = numeroEmpleado;
    }

    /**
     * Metodo que regresa el nombre del departamento asignado al empleado
     * @return Regresa el departamento
     */
    public String getDepartamento() {
        return departamento;
    }

    /**

```

```

    * Metodo que le asigna departamento a un empleado
    * @param departamento Nombre de departamento
    */
    public void setDepartamento(String departamento) {
        this.departamento = departamento;
    }

    /**
     * Metodo que regresa el puesto del empleado
     * @return Regresa un String que indica el puesto
     */
    public String getPuesto() {
        return puesto;
    }

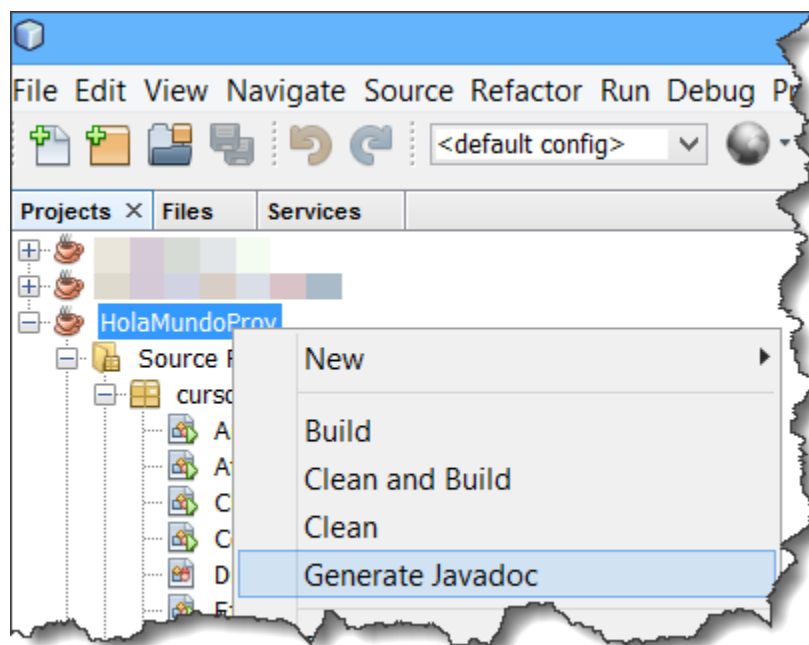
    /**
     * Metodo que establece el puesto del empleado
     * @param puesto Indica el puesto del empleado
     */
    public void setPuesto(String puesto) {
        this.puesto = puesto;
    }
}

```

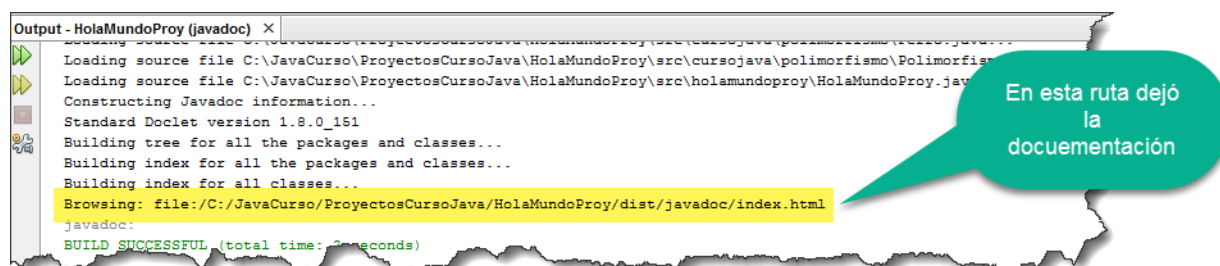
Una vez que ya tengamos todas nuestras Clases, interfaces, etc documentadas, procedemos a crear documentación de Java. En NetBeans se hace de la siguiente manera.

Generar javaDoc en NetBeans

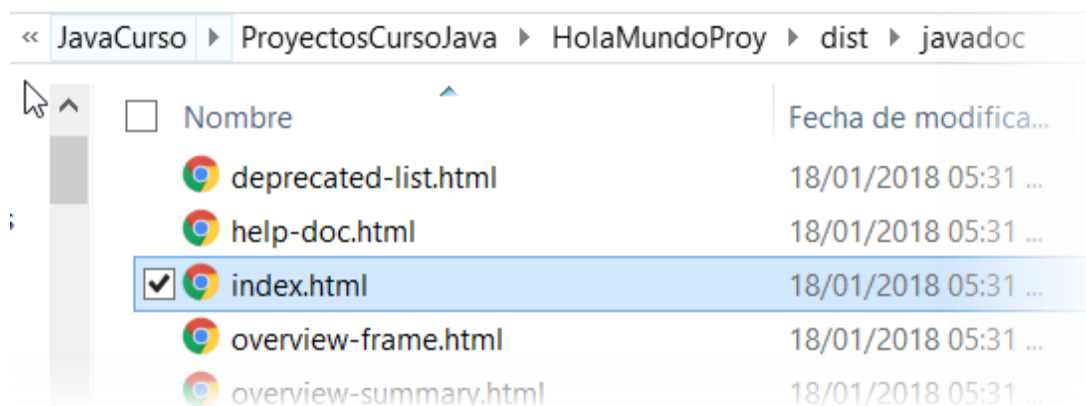
Clic derecho sobre nuestro proyecto y seleccionamos la opción “Generate Javadoc”.



El editor comienza a generar la documentación de nuestro proyecto.



Abrimos la ruta y entramos a la carpeta javadoc (si no aparece la ruta, buscamos directamente en la carpeta del proyecto), vemos que se crearon varios archivos, buscamos el archivo "index.html" y lo abrimos con un navegador web.



Listo, ya tenemos la documentación Java de nuestro proyecto, el comando “javadoc” detectó todos los objetos que creamos en nuestro proyecto.

Creó sus páginas de documentación, realizó el orden por paquetes, listado de clases y contenidos.

The screenshot shows a web browser displaying the Javadoc documentation for the 'Empleado' class. The browser's address bar shows the file path: 'file:///C:/JavaCurso/ProyectosCursoJava/HolaMundoProy/dist/javadoc/index.html'. The page has a sidebar on the left with a tree view of packages and classes. The main content area is titled 'Class Empleado' and shows the class hierarchy: 'java.lang.Object', 'cursojava.Persona', and 'cursojava.Empleado'. It also displays the class declaration: 'public class Empleado extends Persona'. Below this, there is a description of the class and a 'Constructor Summary' section. The 'Constructor Summary' section lists two constructors: 'Empleado()' and 'Empleado(java.lang.String nombre, int edad, char sexo, int numeroEmpleado, java.lang.String departamento, java.lang.String puesto)'. The 'Empleado()' constructor is highlighted with a yellow background.

Ya podemos comenzar a revisar nuestra documentación.

This image is a close-up of the 'Constructors' section from the Javadoc documentation. It shows a table with two columns: 'Constructor and Description'. The first row shows the 'Empleado()' constructor with the description 'Metodo constructor por defecto'. The second row shows the 'Empleado(java.lang.String nombre, int edad, char sexo, int numeroEmpleado, java.lang.String departamento, java.lang.String puesto)' constructor with the description 'Metodo constructor parametrizado, permite crear un empleado a partir de los parametros introducidos'. A red box highlights the 'Empleado' link in the second row, and a green dashed arrow points from the 'Empleado()' link in the first row to the 'Empleado' link in the second row.

Al dar clic en el link del método nos manda directo a su especificación.

Empleado

```
public Empleado(java.lang.String nombre,  
                int edad,  
                char sexo,  
                int numeroEmpleado,  
                java.lang.String departamento,  
                java.lang.String puesto)
```

Metodo constructor parametrizado, permite crear un empleado a partir de los parametros introducidos

Parameters:

nombre - Nombre del empleado

edad - Edad del empleado

sexo - Sexo del empleado en formato H o M

numeroEmpleado - Numero de empleado

departamento - Departamento donde trabajara el empleado

puesto - Puesto que ocupara el empleado dentro de la empresa

EJERCICIO

Crea una clase denominada **CalculadoraEjemplo** que tenga los atributos operando1, operando2, resultado y métodos que implementen las operaciones básicas (Suma, Resta, Multiplicación y División). Documenta esta clase conforme a los **estándares JavaDoc** y comprueba cómo se visualiza la documentación.